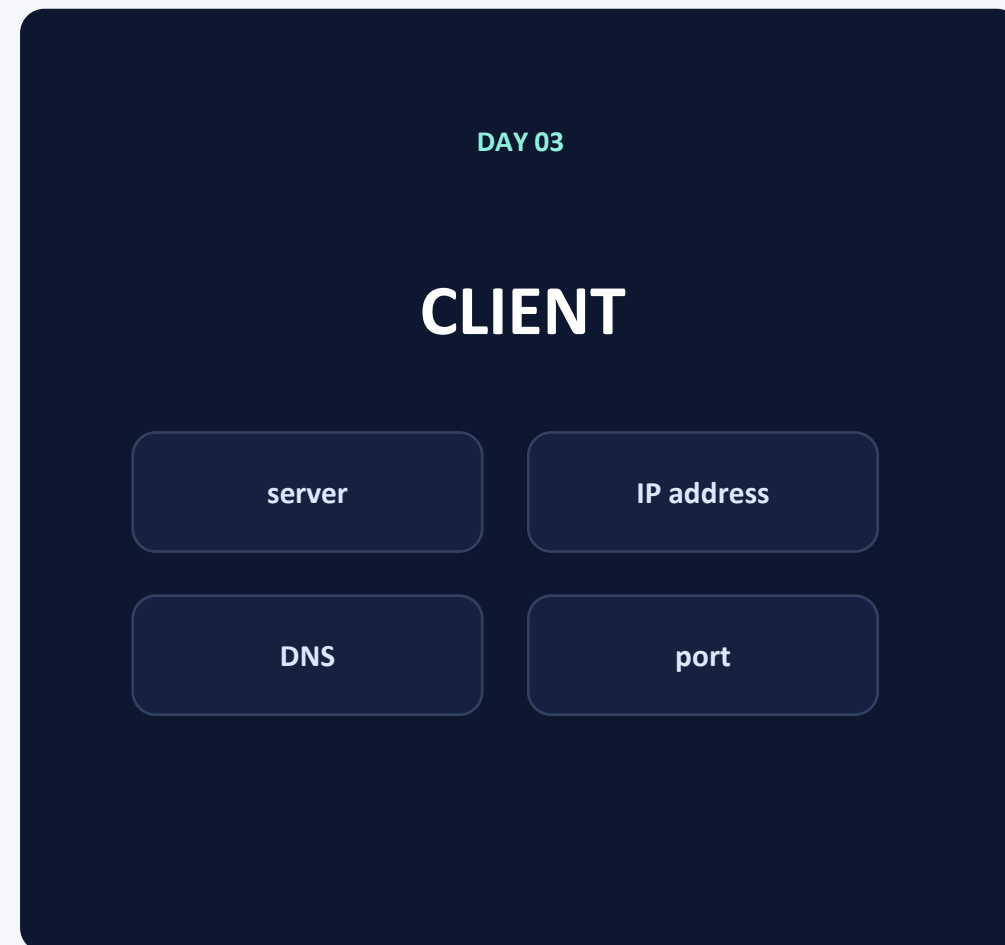


# Интернет, клиент и сервер

Проследить путь сетевого запроса: от клиента через DNS и IP к нужному серверному порту и обратно.



ЦЕЛЬ ДНЯ

# Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Проследить путь сетевого запроса: от клиента через DNS и IP к нужному серверному порту и обратно.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

# Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

## Как устроен компьютер

Что уже есть в нашей модели.



СЕГОДНЯ

## Интернет, клиент и сервер

Вчера: Как устроен компьютер. Сегодня мы добавляем следующий слой: интернет, клиент и сервер.

ЗАЧЕМ

# Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

**Без этой модели трудно понимать браузер, API, серверы и почти всё, что появится дальше в веб-разработке.**

Вопрос себе

**Какую реальную проблему решает сегодняшняя тема?**

Ответь до следующего слайда.

# Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

## БЫТОВОЙ ОБРАЗ

**Интернет похож на доставку: домен — имя получателя, DNS — справочник адресов, IP — адрес здания, порт — номер кабинета, протокол — правила передачи.**

## ПЕРЕВОД В IT

**client**

программа или устройство, которое запрашивает услугу

**server**

программа или компьютер, который обрабатывает запросы

**IP address**

сетевой адрес устройства для доставки данных

# Слой 1: client • server • IP address

Сначала смысл, затем профессиональный термин.

**client**

программа или устройство, которое запрашивает услугу

**server**

программа или компьютер, который обрабатывает запросы

**IP address**

сетевой адрес устройства для доставки данных

## Слой 2: DNS • port • protocol

Сначала смысл, затем профессиональный термин.

**DNS**

система, которая преобразует доменное имя в IP-адрес

**port**

номер конкретной сетевой службы на устройстве

**protocol**

согласованный набор правил обмена данными

## Слой 3: domain • router

Сначала смысл, затем профессиональный термин.

**domain**

читаемое человеком имя интернет-ресурса

**router**

устройство, направляющее пакеты между сетями

Не заучивай формулировку. Придумай свой пример использования каждого термина.

## Слой 4: packet • latency

Сначала смысл, затем профессиональный термин.

**packet**

небольшая порция данных, передаваемая по сети

**latency**

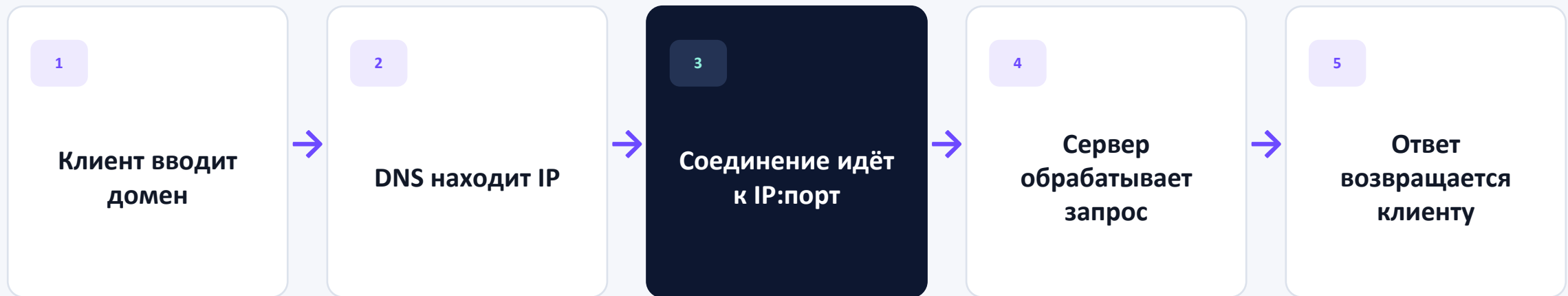
задержка между отправкой запроса и получением ответа

Не заучивай формулировку. Придумай свой пример использования каждого термина.

## СХЕМА

# Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

# Первый минимальный пример

Смотрим только на одну основную идею.



day03\_step1

```
client = "браузер"  
server = "example.com"  
  
print(client, "отправляет запрос на", server)  
print(server, "возвращает ответ")
```

## ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

# Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
client = "браузер"
```

Сохраняем значение или результат вычисления под понятным именем.

```
server = "example.com"
```

Сохраняем значение или результат вычисления под понятным именем.

```
print(client, "отправляет запрос на",
```

Показываем результат, чтобы увидеть состояние программы.

```
print(server, "возвращает ответ"))
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

# Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
client = "браузер"  
server = "example.com"  
  
print(client, "отправляет запрос на", server)  
print(server, "возвращает ответ")
```

РЕЗУЛЬТАТ

браузер отправляет запрос на  
example.com  
example.com возвращает ответ

ИЗМЕНИЛИ

# Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.

```
domain = "shop.example"  
ip = "203.0.113.10"  
port = 443  
  
print(domain, "->", ip, "порт", port)
```

changed

?

**Что появится в  
результате?**

Сформулируй ответ словами.

ОТВЕТ

# Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

shop.example -> 203.0.113.10 порт 443

ПОЧЕМУ

**Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.**

Сравни с первым примером: что изменилось?

# Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.

```
client = "браузер"  
print(browser, "отправляет запрос")
```

wrong

СТОП

**Что здесь может сломаться  
или ввести нас в  
заблуждение?**

Сначала найди проблему сам.

# Почему возникает проблема

Переводим техническую причину на обычный язык.

**browser не определён. Строка «браузер» хранится в переменной client, значит использовать нужно имя client.**

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

## ИСПРАВЛЕНИЕ

# Исправленный вариант

Меняем только то, что действительно было причиной.

```
client = "браузер"  
print(client, "отправляет запрос")
```

fixed

## ПРАВИЛО

**Не переписывай всё решение. Найди минимальное осмысленное исправление.**

# Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

# Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

## ЗАДАЧА

Разложить открытие `shop.example` по шагам: домен → DNS → IP:443 → запрос → ответ.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

# Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 Пользователь вводит `shop.example`.
- 2 DNS возвращает IP сервера.
- 3 Клиент соединяется с портом 443.
- 4 Отправляет запрос по согласованному протоколу.
- 5 Сервер возвращает ответ.

# Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
domain = "shop.example"  
ip = "203.0.113.10"  
port = 443  
  
print("1. Домен:", domain)
```

solution\_part1

## Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

## Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.

```
domain = "shop.example"
ip = "203.0.113.10"
port = 443

print("1. Домен:", domain)
print("2. DNS ->", ip)
print("3. Соединение ->", f"{ip}:{port}")
print("4. Запрос")
print("5. Ответ")
```

solution\_final

# Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

## РЕЗУЛЬТАТ

1. Домен: shop.example
2. DNS -> 203.0.113.10
3. Соединение -> 203.0.113.10:443
4. Запрос
5. Ответ

## ГЛАВНОЕ

**Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.**

ПОПРОБУЙ САМ

# Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

**Объясни путь запроса к `api.example.com:8080` обычными словами и подпиши роль DNS, IP и порта.**

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

# Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

**Домен отвечает «как найти адрес», порт — «к какой службе на адресе обратиться».**

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

# Один из правильных вариантов

Сравни логику, а не количество строк.



practice\_solution

```
domain = "api.example.com"  
ip = "198.51.100.7"  
port = 8080  
print(domain, "-> DNS ->", ip, "-> порт", port)
```

# Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

## РЕЗУЛЬТАТ

api.example.com -> DNS -> 198.51.100.7 -> порт 8080

## Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

## МИНИ-ПРОЕКТ

# Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую законченную задачу.

**ЗАДАНИЕ**

**Собери «интернет-посылку»: клиент, домен, IP, порт, запрос и ответ.**

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

# Разбор мини-проекта

Вот один рабочий путь.



```
route = ["клиент", "DNS", "IP", "порт", "сервер", "ответ"]
for point in route:
    print("->", point)
```

mini\_project

# Что должно получиться

Связываем итог с задачей и проверяем смысл.

## РЕЗУЛЬТАТ

- > клиент
- > DNS
- > IP
- > порт
- > сервер
- > ответ

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

# Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01 Путать домен и IP.

02 Считать DNS самим сервером сайта.

03 Забывать, что на одном IP может быть несколько сервисов.

04 Считать порт физическим разъёмом.

# Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «client» и какую проблему это понятие помогает описать?

2

Чем «client» отличается от «DNS»? Объясни без копирования определения.

# Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

## ОТВЕТЫ

# Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

**1**     **client — программа или устройство, которое запрашивает услугу.**

**2**     client: программа или устройство, которое запрашивает услугу. DNS: система, которая преобразует доменное имя в IP-адрес.

## ОТВЕТЫ

# Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

**3    браузер отправляет запрос на `example.com`  
`example.com` возвращает ответ**

**4    `browser` не определён. Строка «браузер» хранится в переменной `client`, значит использовать нужно имя `client`.**

ИТОГ

# Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ **Проследить путь сетевого запроса: от клиента через DNS и IP к нужному серверному порту и обратно.**
- ✓ **Читать и объяснять пример темы «Интернет, клиент и сервер».**
- ✓ **Находить типичную ошибку и формулировать причину.**
- ✓ **Решать похожую задачу по шагам, а не угадывать готовый код.**

ДАЛЬШЕ

# Что будет завтра

Связываем сегодняшний результат со следующим шагом.

## Как работает Web

Завтра: Как работает Web. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.